

FitBuddy

AI Fitness Plan Generator using Gemini Models

Project Type	AI-powered web application
Backend	FastAPI
AI Layer	Google Gemini models
Database	SQLite + SQLAlchemy
Frontend	HTML + Jinja2 + CSS
Server	Uvicorn

1. Project Overview

FitBuddy is a web-based application that uses AI to generate personalized workout plans and nutrition or recovery tips based on a user's fitness goal, age, weight, and preferred workout intensity. The system generates a structured 7-day plan and allows feedback-based refinement.

User information and generated plans are stored in SQLite through SQLAlchemy, and an administrator/coach can review users and their original or updated plans.

2. Project Scenarios

Scenario 1 – Personalized Plan

The user enters name, user ID, age, weight, fitness goal and workout intensity to receive a personalized 7-day schedule.

Scenario 2 – Feedback Update

The user submits feedback such as adding cardio, yoga or rest days. The existing plan and feedback are used to create an updated plan.

Scenario 3 – Nutrition / Recovery Tip

A concise tip is generated according to the selected fitness goal.

Scenario 4 – Admin / Coach View

An administrator or coach can view registered users and their original and updated plans.

3. Technical Architecture

Layer	Technology / Responsibility
Frontend	HTML, CSS and Jinja2 templates

Backend	FastAPI for routing and form processing
AI Layer	Google Gemini APIs for workout generation, updates and nutrition tips
Database	SQLite with SQLAlchemy ORM
Server	Uvicorn ASGI server

4. Project Milestones

Milestone 1: Model Selection and Architecture

Select the AI model, define architecture and prepare the environment.

Milestone 2: Core Functionalities

Develop workout generation, nutrition tips, feedback updates and database storage.

Milestone 3: routes.py Development

Connect forms, AI functions, database functions and templates through FastAPI routes.

Milestone 4: Frontend Development

Design responsive pages and integrate dynamic Jinja2 templates.

Milestone 5: Deployment

Prepare the local environment, configure variables and verify the application.

5. AI Model Selection

The supplied project document describes experimentation with T5, BART, DistilBERT and Bloom, followed by selection of Gemini models for production.

- Gemini 1.5 Pro – structured 7-day workout generation and plan updating.
- Gemini Flash – fast nutrition or recovery tips.

The document gives API integration, structured formatting, accuracy, scalability and fast feedback-based revision as reasons for the selection.

6. Core Functions and Files

Function	File	Purpose
generate_workout_gemini()	gemini_generator.py	Generates a personalized 7-day workout plan.
generate_nutrition_tip_with_flash()	gemini_flash_generator.py	Generates a concise nutrition/recovery tip.
update_workout_plan()	updated_plan.py	Creates a revised plan using original plan + feedback.
save_user(), save_plan(), update_plan()	database.py	Stores and updates user and plan information.
get_original_plan(), get_user()	database.py	Retrieves stored information.

The workout output includes warm-up, main workout details such as exercises/sets/repetitions, and cooldown or recovery guidance.

7. FastAPI Routes

Route	Method	Purpose
/	GET	Displays the home page and input form.
/generate-workout	POST	Receives user details, generates workout and nutrition tip, stores results and renders.
/submit-feedback	POST	Receives feedback, updates the existing plan and stores the revision.
/view-all-users	GET	Displays users and their original/updated plans.

8. Frontend Pages

index.html

Main input form for name, user ID, age, weight, goal and intensity.

result.html

Displays user information, the 7-day plan, nutrition tip and feedback form.

all_users.html

Admin page for user records and original/updated plans.

9. UI and Jinja2

The document describes a fitness-oriented interface with a gym-themed background, Roboto font, bold typography, centered layouts, responsive media queries, styled buttons, rounded input/result blocks and a dark background.

FastAPI's Jinja2Templates renders dynamic content. Forms bind to backend Form(...) parameters, while values such as username, goal, intensity, workout_plan and nutrition_tip are displayed dynamically.

10. Database and Data Flow

User information and AI-generated plans are stored using SQLite and SQLAlchemy ORM. Feedback-based updates are stored separately so the original plan can be preserved.

User Input → FastAPI Route → Gemini Workout Generation → Gemini Nutrition Tip → SQLite Storage → Jinja2 Result Page → Feedback → Gemini Plan Update → Database Update → Updated Result

11. Environment Setup

The project uses Python, pip, a virtual environment, FastAPI, Uvicorn, Jinja2, SQLAlchemy, python-multipart and Google Generative AI support.

Windows setup commands:

```
python -m venv fitbuddy-env
fitbuddy-env\Scripts\activate
pip install fastapi uvicorn jinja2 sqlalchemy python-multipart google-generativeai
uvicorn app.main:app --reload
```

Environment variable:

```
GOOGLE_API_KEY=your_gemini_api_key_here
```

12. Local Testing

Run the application with Uvicorn and open <http://127.0.0.1:8000>. FastAPI interactive documentation is available at </docs>.

Testing checklist:

- Open the home page.
- Enter user details.
- Generate the 7-day workout plan.
- Verify the nutrition/recovery tip.
- Submit feedback and verify the updated plan.
- Open the admin view and verify stored records.
- Open </docs> and inspect the endpoints.

13. Expected User Experience

Home Page

Simple form for personal and fitness information.

Personalized Plan

User details plus the generated plan.

Workout Plan

Day-wise focus, exercises, sets/repetitions or duration, rest and basic warm-up/cool-down guidance.

Nutrition Tip

Short goal-aligned nutrition or recovery advice.

Feedback

User can request changes to the existing plan.

Admin View

Registered users and original/updated plans can be reviewed.

14. Project Structure

```
FitBuddy_Project/  
├── app/  
│   ├── main.py  
│   ├── routes.py  
│   ├── database.py  
│   ├── gemini_generator.py  
│   ├── gemini_flash_generator.py  
│   ├── updated_plan.py  
│   └── templates/  
│       ├── index.html  
│       ├── result.html  
│       └── all_users.html  
├── static/  
│   └── images/  
├── requirements.txt  
├── .env  
└── README.md
```

15. Conclusion

FitBuddy demonstrates how generative AI can be combined with FastAPI, Jinja2 and SQLite to provide personalized fitness planning. The project covers AI-based workout generation, nutrition/recovery guidance, feedback-driven updates, persistent storage and an administrative view.

Documentation prepared from the supplied FitBuddy project document.